PROJECT :: The xargs Utility in Java

Project Objective

Your task is to implement a simplified version of the Unix `xargs` utility in Java. This project will help you gain hands-on experience with file input/output, command-line argument parsing, and string manipulation—fundamental skills in systems programming and process management. All other projects in this course will required these skills. This is designed to be a low stress refresher for ideas that you should have covered in your course on Linux as well as give you more practice working with Linux and the command line.

It's recommended that you work your way through this **tutorial on xargs (https://csus.instructure.com/courses/21989/pages/tutorial-the-xargs-utility)** before you begin programming this project.

Learning Objectives

By completing this project, you will achieve the following learning objectives:

- **Practice Command-Line Argument Parsing**: Develop skills in parsing and handling command-line arguments in Java, including the use of options and flags to modify program behavior.
- **Handle Input and Output in Unix-like Systems**: Gain proficiency in reading input from standard input, manipulating strings, and building command lines based on input data.
- **Implement Input Sanitization for Security**: Understand the importance of input sanitization by filtering specific special characters to prevent command injection attacks, reinforcing secure coding practices.
- **Execute External Commands**: Learn how to use `ProcessBuilder` to execute external commands based on dynamically generated input, a core aspect of the `xargs` utility.
- **Reinforce Knowledge of Special Characters in Linux**: Refresh and strengthen your understanding of how special characters like `;`, `&`, `|`, and others are used in Linux, and how they can impact command execution.
- **Simulate Command-Line Behavior**: Gain experience in simulating the behavior of Unix command-line utilities by building a simplified version of `xargs`, which combines multiple inputs into a single command execution.

These objectives focus on the key aspects of the `xargs` utility, emphasizing input handling, command execution, and security considerations, while providing a solid foundation for further exploration in Unix-like environments.

Background

The `xargs` utility reads items from standard input (or from a file) and executes a command using those items as arguments. This allows you to build and execute command lines from dynamically generated input. The utility is commonly used in combination with other Unix tools like `find` and `grep`.

In this project, you will recreate key features of `xargs`, including:

- Reading input from standard input.
- Splitting input on whitespace and treating each word as a separate argument.
- Building and executing command lines using the input as arguments.
- Handling basic command-line options.

Requirements

1. **Input Handling**: Your program should read input from standard input. Each word of input should be treated as a separate argument unless otherwise specified by the `-I` option.
2. **Command Execution**: The program should construct a command line using the input it reads and execute the command with those arguments. Use `ProcessBuilder` to handle command execution.
3. **Basic Options**:
    - `-n <num>`: Specify the maximum number of arguments to pass to the command at one time.
    - `-I <replace>`: Replace occurrences of `<replace>` in the command with input read from standard input.
    - `-t`: Print the entire constructed command line to standard output before executing it.
    - `-r`: Do not run the command if no input is provided.
4. **Error Handling**: The program should handle errors gracefully, including cases where the command fails or where the input cannot be processed as expected. Ensure that empty input is ignored if the `-r` option is specified.
5. **Security Considerations**:
    - **Sanitize Input**: Your program must filter out specific special characters from the input before it is used in command execution. The characters to filter are:
        - `;` - Used to terminate commands and start new ones.
        - `&` - Used to run commands in the background.
        - `|` - Used to pipe the output of one command to another.
        - `>` - Used to redirect output to a file.
        - `<` - Used to redirect input from a file.
        - `*` - Wildcard character that can match filenames.

    - `?` - Wildcard character that matches a single character.
    - `(` and `)` - Can be used in subshells.
    - `$` - Used to reference shell variables.
  - **Handle Special Characters**: Ensure that your implementation handles special characters and spaces appropriately in input processing.
  - **Avoid Direct Shell Execution**: Ensure that the command execution is secure and only the intended command and arguments are executed, without invoking unintended shell operations.

Introduction to ProcessBuilder

In this section, we will explore the `ProcessBuilder` class in Java, which you'll use in your `xargs` project. `ProcessBuilder` is a key tool in Java for creating and managing processes, allowing you to execute external commands and control their input and output.

`ProcessBuilder`: Executing a Command

The `ProcessBuilder` class provides a way to create and manage operating system processes in Java. It is used to execute external commands and control their input and output streams.

Key Points:

- **Command and Arguments**: `ProcessBuilder` takes a list of strings as arguments, where the first string is the name of the command, and the subsequent strings are the arguments.
- **Environment Variables**: You can modify the environment variables for the process using the `environment()` method.
- **Redirection**: `ProcessBuilder` allows you to redirect the input, output, and error streams of the process using methods like `redirectInput()`, `redirectOutput()`, and `redirectError()`.
- **Inherit IO**: If you want the process to use the same I/O streams as the current process, you can use the `inheritIO()` method.

Example Usage:

```
import java.io.IOException;

public class MyXargsExample {
    public static void main(String[] args) {
        ProcessBuilder pb = new ProcessBuilder("ls", "-l", "/home");
        pb.inheritIO(); // Make the subprocess use the same I/O as the parent process

        try {
            Process process = pb.start();
            process.waitFor(); // Wait for the process to finish
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Handling Input and Output with ProcessBuilder

`ProcessBuilder` allows you to manage the input and output of the process it creates. You can capture the output of a command, send input to it, or redirect it to a file. The following example will capture the output from the command that's passed in and you should see that output printed on the command line. Note that the program above does not show you the output from the "ls" command. Note that the above command also shows you the output, but this version prints it line by line with some additional text to show you how it's being processed.

Example: Capturing Output

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class MyXargsExample {
    public static void main(String[] args) {
        ProcessBuilder pb = new ProcessBuilder("ls", "-l", "/home");

        try {
            Process process = pb.start();
            BufferedReader reader = new BufferedReader(new InputStreamReader(process.getInputStream()));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println("CAPTURED:" + line);
            }
            process.waitFor();
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Summary

- `ProcessBuilder` is used to create and manage processes in Java. It allows you to execute external commands and control their input/output.
- **Handling Input and Output**: `ProcessBuilder` provides methods to redirect or capture the input and output streams of the process it creates.
- **Executing Commands Securely**: By sanitizing input and controlling the execution environment, you can use `ProcessBuilder` to execute commands securely, which is essential for your `xargs` project.

This foundational understanding of `ProcessBuilder` will enable you to implement the command execution features required for the project. While we won’t be covering process management in depth in this networking course, you will encounter these concepts again when we build a web server. For now, this should give you a solid starting point for working with external processes in Java.

Java Compilation and Execution Instructions

Compilation

Your code will be graded using the latest LTS version of Java, but no features beyond Java 11 will be expected. In other words, you may use any version of the JDK that is at least version 11 to work on your project. To compile your `xargs` program, use the following command:

```
javac MyXargs.java
```

This command compiles your `MyXargs.java` file into a `MyXargs.class` file . However, since you're using Java 11's single source file execution format, you can directly run the program without a separate compilation step.

Execution

After ensuring that your code is compatible with Java 11, you should test it using the following execution method:

```
java MyXargs.java
```

This command directly runs your `MyXargs.java` file.

To ensure that your file is compatible with Java 11's single source executable format, make sure your entire program is contained within a single `.java` file. This file should define only one public class, which must have the same name as the file itself. Since this project is relatively simple and only requires one class, you won't need to worry about managing multiple classes or files. Keeping everything in one class not only meets the requirements for Java 11's single source format but also simplifies your development and testing process.

Usage Example

If the program is run without sufficient arguments or with an incorrect configuration, it should display a usage message. Here's how that should look:

```
java MyXargs.java
```

Expected Output:

```
Usage: java MyXargs.java [-n num] [-I replace] [-t] [-r] command
```

This message should be displayed anytime the user fails to provide the correct options or arguments. It helps users understand the required and optional parameters for running your program.

Example Run

Here's an example of how your program might be run, along with the expected behavior:

1. **Basic Execution Example**:

```
echo "file1.txt file2.txt file3.txt" | java MyXargs.java -t rm
```

Expected Behavior: Your program should output the constructed command line (`rm file1.txt file2.txt file3.txt`) before executing it. The files `file1.txt`, `file2.txt`, and `file3.txt` would then be removed.

2. **Using the `-n` Option**:

```
echo "file1.txt file2.txt file3.txt" | java MyXargs.java -n 1 -t rm
```

Expected Behavior: Your program should print and execute the `rm` command for each file individually:

```
+ rm file1.txt
+ rm file2.txt
+ rm file3.txt
```

3. **Using the `-I` Option**:

```
echo "apple orange banana" | java MyXargs.java -I {} -t echo "Fruit: {}"
```

Expected Behavior: Your program should print and execute the `echo` command with all fruit substituted into the placeholder:

```
+ echo Fruit: apple orange banana
Fruit: apple orange banana
```

4. **Using the `-r` Option**:

```
echo "" | java MyXargs.java -r -t rm
```

Expected Behavior: If the input is empty, the program should not execute the `rm` command, and nothing should be printed.

Compare this to the following:

```
echo "" | java MyXargs.java -t rm
```

Expected Behavior: If the input is empty, but -r is not included, then the program should execute the `rm` command.

```
+ rm
rm: missing operand
Try 'rm --help' for more information.
```

Make sure your implementation matches these expectations to ensure consistent and correct behavior. This guidance will help ensure your code compiles and runs as expected in a standardized

Java environment.

Grading Criteria

Your project will be evaluated based on the following criteria:

1. **Correctness and Testing (70%)**:
   - Does the program behave as expected when tested against grading test cases?
   - Does it correctly implement all required features and options (`-n`, `-I`, `-t`, `-r`)?
   - Is the program robust against various input scenarios, including edge cases?
   - Does the program handle input and output appropriately, and does it correctly sanitize and execute commands?
2. **Code Quality and Functional Decomposition (20%)**:
   - Is the code well-structured and modular, with clear functional decomposition?
   - Are functions properly broken down into smaller, manageable tasks rather than being overly complex?
   - Are variables and functions named appropriately, following standard coding conventions?
   - Is the code adequately documented with comments that explain the purpose of each section?
3. **Compilation and Execution (10%)**:
   - Does the program compile without **errors** or **warnings** using the specified compilation command?
   - Does the program run as expected in the target environment (Ubuntu)?
   - If the program fails to compile or execute correctly, it cannot receive more than 10% of the grade, which will be based purely on code quality. No other test cases will be considered for code that fails to compile or execute.

Important Note: You will have access to a GradeScope based checker tool that will help you verify whether your program compiles and runs. This tool will not run all grading test cases, it's primarily intended to make sure that you are properly obeying Java 11 single source executable rules. Make sure to test your code with this tool before submission to ensure that it meets the basic requirements.

This grading rubric emphasizes the importance of correctness, functional decomposition, and clean, modular code. Programs that do not compile or run successfully can only earn points for code quality, with a maximum of 10% of the total grade.

Submission Details

You will be submitting a single Java file for this project. Most likely you will submit with GradeScope. Details will be posted next week.

Release History

Just a friendly reminder: Keeping up with the latest version of assignments is key! We might update them now and then to iron out any bugs or to make things clearer. Whenever there's an update, we'll let you know through a Canvas announcement, so please make sure you're set up to receive those notifications. Embrace the dynamic nature of programming, where adapting to evolving requirements is part of the adventure. If you spot any bugs in the assignments, feel free to send me a bug report. It's a great way to help out and contribute to our course community!

V003 : Added missing input and some additional text to the second example.
V002 : Edited some incorrect behaviors and add some more detail.
V001 : Initial release, expect a few bugs.